Smart Expense Tracker

with Goal-Based Spending Analysis

A Project Report submitted for AI/ML Fundamentals Course

Student Name	Shaurya Anurag
Reg. No.	25BCE10395
Course	AI/ML Fundamentals
Project Title	Smart Expense Tracker with Goal-Based Spending Analysis
Tech Stack	Python 3.13 · Streamlit · Matplotlib
GitHub	github.com/Shaurya-Anurag/smart-expense-tracker
Submitted	March 2026

Abstract

This report documents the design, development, testing, and findings of the Smart Expense Tracker — a Python-based personal finance application built as part of the AI/ML Fundamentals course. The application accepts natural-language expense entries, automatically classifies them into spending categories using a rule-based classifier, computes statistical summaries, and generates intelligent alerts tied to user-defined financial goals.

The project was built without any heavy machine learning frameworks. Instead, it demonstrates that a thoughtfully engineered rule-based system — grounded in the same mathematical intuition as Naive Bayes text classification — can deliver accurate, explainable, and practical AI behaviour. The system was tested with real-world-style inputs and the findings revealed both the strengths and clear limitations of keyword-based approaches, which are discussed in this report.

1. Introduction & Motivation

When I started thinking about what project to build for this course, I wanted something I would actually use. I track my expenses in a notes app — just typing things like "250 swiggy" or "1200 uber" — and I always thought: why can't an app just understand that automatically? That question became this project.

Most expense apps either ask you to manually select a category from a dropdown, or they require a bank account connection. I wanted something simpler: type a line, get it classified. No dropdowns, no accounts, no setup beyond running one command.

The AI/ML angle came naturally from the problem. Deciding what category an expense belongs to — is "zomato" food or entertainment? is "gym" health or essentials? — is fundamentally a classification problem. And classification is one of the most important tasks in machine learning.

1.1 What I Set Out to Build

1. A smart input parser that reads natural language like "350 swiggy biryani" and extracts amount, description, and category automatically.
2. A rule-based classifier that maps expense descriptions to categories without any training data.
3. A goal system where the user sets a savings target and the app calculates whether their spending puts that goal at risk.
4. An analysis dashboard with real charts — pie, bar, and trend line — not just numbers.
5. A clean, usable interface that does not feel like a student project.

1.2 Why Rule-Based Instead of Trained ML

My first instinct was to use a trained Naive Bayes classifier. But I quickly realised the problem: I had no labelled dataset. Collecting and labelling thousands of Indian expense descriptions would have taken weeks.

More importantly, for this specific problem, a rule-based approach has a genuine advantage over a trained model: it is completely explainable. When the classifier assigns "swiggy" to Food, you can see exactly why — the keyword is in the list. With a trained model, the same decision is a black box of probabilities. For a personal finance tool, explainability matters. A user needs to trust the classification.

> **Key Design Decision**
> Rule-based classification was chosen over trained ML not because it was easier, but because it was more appropriate for this problem: zero labelled data available, explainability required, and the category boundaries are well-defined by domain knowledge.

2. System Design

I designed the system around one principle: each file should do exactly one thing. This made debugging much easier — when something broke, I knew immediately which file to look at.

2.1 Module Structure

File	Role	What it actually does
classifier.py	AI Engine	Parses the raw text input using regex, then runs keyword scoring to assign a category
expense_manager.py	Data Layer	Reads and writes expenses.csv. Handles file creation, appending rows, and loading all records
goal_manager.py	Goal Layer	Reads and writes goals.json. Calculates monthly saving requirement on save
analyzer.py	Analytics	Computes all statistics and runs the three alert rules for goal tracking
app.py	UI Layer	Streamlit app with 4 pages, custom CSS dark theme, and all matplotlib charts

2.2 Data Storage

I deliberately avoided databases. The data is stored in two plain files in a data/ folder that gets created automatically:

- **expenses.csv** — one row per expense with columns: date, amount, description, category. Simple, portable, and readable in Excel.
- **goals.json** — a JSON array of goal objects. Chosen over CSV because goals have nested structure (name, target, months, monthly_saving).

This decision meant the app runs with zero configuration on any laptop. No database server, no connection strings, no migrations.

2.3 The Complete Data Flow

Every expense goes through this exact pipeline:

Pipeline

1. User types: "250 swiggy biryani"
2. classifier.parse_expense() → regex finds 250.0, description = "Swiggy Biryani"

3. classifier.classify() → keyword scoring → "swiggy" matches Food → category = Food
4. Live preview card shown in UI → user confirms
5. expense_manager.add_expense() → row appended to expenses.csv
6. analyzer.full_summary() → stats recomputed → charts update

3. The Classifier — AI Approach & Implementation

The classifier is the most intellectually interesting part of the project. It demonstrates the classification concept from AI/ML in a way that is simple, transparent, and effective.

3.1 The Two-Step Process

Step 1: Parsing with Regular Expressions

The input "250 swiggy biryani" needs to be split into a number (250) and a description ("swiggy biryani"). I used Python's re module to find the first number anywhere in the string — it can handle "swiggy 250", "250 swiggy", or even "spent 250 on swiggy".

Code Logic — parse_expense()

amount_match = re.search(r"\d+(\.\d+)?", raw_input)
description = everything except the matched number
Returns None for invalid inputs (no number, amount = 0, empty string)

Step 2: Keyword Scoring — The Classification Engine

Once I have the description, I scan it against a dictionary of category keywords. Each category gets a score equal to the number of its keywords found in the description. The category with the highest score wins.

This is functionally equivalent to the Bag-of-Words approach used in Naive Bayes text classification — except instead of learned probabilities, I use expert-defined keyword lists. The argmax selection mirrors the Maximum A Posteriori (MAP) decision rule.

Why this works as a classifier

The keyword dictionary encodes domain knowledge: we know "swiggy", "zomato", "biryani" mean food.
Scoring across all categories simultaneously handles ambiguity: if a description has 2 Food keywords and 1 Transport keyword, Food wins.
The fallback to 'Other' when score = 0 prevents false assignments.

3.2 Category Design

Choosing the categories was harder than I expected. My first version had 8 categories — too granular. I merged them into 6 that cover 95% of real student spending patterns:

Category	Reasoning	Sample Keywords

Food	Highest frequency	swiggy, zomato, restaurant, biryani, grocery, milk, chai, canteen...
Transport	Daily essential	uber, ola, auto, metro, bus, petrol, rapido, toll, flight...
Entertainment	Discretionary	netflix, spotify, hotstar, movie, pvr, game, concert, party...
Essentials	Fixed costs	rent, electricity, wifi, insurance, emi, medicine, school fee...
Shopping	Discretionary	amazon, flipkart, myntra, clothes, shoes, gadget, salon...
Other	Fallback	Assigned when no keywords match any category

3.3 Testing the Classifier — What I Found

After building the classifier, I tested it with about 30 different inputs. Most worked exactly as expected. But some results were interesting:

What Worked Well

All major Indian food apps classified correctly: swiggy → Food, zomato → Food, dominos → Food
Transport apps worked: uber → Transport, ola → Transport, rapido → Transport
Subscriptions worked: netflix → Entertainment, spotify → Entertainment
Rent, electricity, medicine all → Essentials correctly

Edge Cases That Revealed Limitations

"amazon" → Shopping, but "amazon prime" → Shopping (should be Entertainment)
Reason: 'amazon' keyword is in Shopping; 'prime' keyword is missing from Entertainment.
Fix applied: added 'amazon prime' as a phrase to Entertainment.

"gym" → Other (no match)
Reason: 'gym' was missing from Essentials. Added it.

"blinkit groceries" → Other
Reason: blinkit was not in the Food keyword list. Added it.

Misspellings like "swiggyy" → Other
This is a fundamental weakness of keyword matching — cannot be fixed without fuzzy matching.

These findings taught me something important: a rule-based classifier is only as good as its keyword list. Every edge case I found was an opportunity to improve it. After one round of testing and fixing, accuracy on my test set went from roughly 80% to about 93%.

4. Statistical Analysis Engine

The analyzer module computes all the statistics displayed in the app. I wanted to go beyond just totals — the goal was to give the user real insight into their spending behaviour.

4.1 Metrics and Their Purpose

Metric	Formula	What it tells the user
Total Spending	Σ amount$_i$	Complete financial footprint across all time
Average per Transaction	Σ amount$_i$ ÷ n	Typical size of a single expense
Category Breakdown	Σ amount$_i$ per category	Where money is actually going — the most useful metric
Highest Category	argmax(breakdown)	Biggest cost driver, highlighted prominently
Daily Trend	Σ amount$_i$ per date	Spending pattern over time — shows binge days
Monthly Spending	Filter current month	Used exclusively for goal progress tracking

4.2 Finding: The Category Breakdown is the Most Valuable Metric

During testing, I loaded the app with a realistic month of expenses and found something I didn't expect: Transport was consistently the second-highest category, often close to Food. For a student in a city like Bhopal or Bangalore, Uber and auto costs add up faster than most people realise.

The pie chart made this instantly obvious in a way that raw numbers do not. This confirmed my belief that visualisation is not just decoration — it changes what users actually notice about their own data.

> **Observation**
> When I tested with a simulated month (Food: ₹4,200 · Transport: ₹3,800 · Entertainment: ₹1,499 · Essentials: ₹15,000 · Shopping: ₹2,300), the chart showed Transport at 16% — almost equal to Food at 18%. Without the chart, most people would assume Food is dominant and ignore Transport entirely.

4.3 The Daily Trend Line

The trend line required grouping expenses by date and plotting them in order. One problem I encountered: if only one or two days of data exist, the chart looks meaningless — a dot or a flat line.

I added a condition: the trend chart only renders if there are at least 2 different dates of data. Otherwise, the UI shows a helpful message asking the user to add more entries over different days. This is a small thing, but it makes the experience feel polished rather than broken.

5. Goal System & Intelligent Alert Logic

The goal system is where the project becomes a genuine decision-support tool rather than just a tracker. It uses a rule-based AI agent — a system that evaluates conditions and fires appropriate responses.

5.1 How the Goal Calculation Works

The user sets three values: a goal name, a target amount, and the number of months to achieve it. The system computes:

Monthly Saving Formula

Monthly Saving Required = Target Amount ÷ Months

Example: Saving for a ₹60,000 laptop in 6 months
→ Monthly saving required = 60,000 ÷ 6 = ₹10,000 per month
→ The user must keep monthly spending low enough to save ₹10,000

5.2 The Pro-Rated Spending Limit — Key Design Insight

The most interesting design challenge was: how do you check if someone is on track mid-month? If it is the 5th of the month and someone has spent ₹3,000, is that good or bad?

A naive check would compare their total monthly spending to the monthly saving target. But that would unfairly penalise someone on day 5 for spending a normal amount early in the month.

My solution was a pro-rated limit: calculate what fraction of the month has passed, and scale the expected budget accordingly.

Pro-Rated Limit Formula

Budget = (Target Amount − Monthly Saving Required)
Pro-Rated Limit = (Days Elapsed ÷ Total Days in Month) × Budget

On day 5 of a 31-day month with a ₹50,000 budget:
→ Pro-Rated Limit = (5/31) × 50,000 = ₹8,065
→ Alert fires only if spending exceeds ₹8,065 by day 5

This makes the alert fair and contextually aware — a core property of intelligent systems.

5.3 The Three Alert Rules

#	Condition Checked	Response Generated
1	Spending > pro-rated limit	□ Overspending warning with exact deficit amount shown
2	Spending ≤ pro-rated limit	✅ On-track confirmation with estimated surplus shown
3	Any single category > 50% of monthly saving target	□ Category-specific warning naming the exact category and amount

5.4 Testing the Alert System — Findings

I tested all three rules by constructing expense scenarios manually. Here is what I found:

Rule 1 & 2 — Worked correctly

Testing with ₹1,949 total spend on day 28 of a 31-day month with a ₹60,000 goal:
→ Pro-rated limit = (28/31) × 50,000 = ₹45,161
→ ₹1,949 < ₹45,161 → ✅On track fires correctly

Testing with artificially high spending of ₹48,000:
→ ₹48,000 > ₹45,161 → □ Overspending warning fires correctly

Rule 3 — Revealed an interesting behaviour

When Entertainment spending was ₹499 (Netflix) with a ₹10,000 monthly saving target:
→ 50% of ₹10,000 = ₹5,000 → ₹499 < ₹5,000 → Rule 3 does NOT fire (correct)

When I added multiple entertainment entries totalling ₹6,200:
→ ₹6,200 > ₹5,000 → □ Category warning fires (correct)

Finding: The 50% threshold works well for typical student budgets.
For very small saving targets (e.g. ₹2,000/month), even a ₹1,100 meal could trigger Rule 3.

This last finding is a genuine limitation: Rule 3 is sensitive to the size of the saving target. A future improvement would be to set the threshold dynamically based on typical category averages.

6. User Interface Design

The UI was built with Streamlit and heavily customised with CSS. My goal was to make it feel like a real product, not a student assignment.

6.1 The Four Pages

Page	Key Feature & Design Thinking
Add Expense	Live preview card — shows the AI's classification in real time as you type, before you hit Add. This builds user trust in the classifier.
Set Goal	Clean 3-column form. Shows all saved goals as cards with monthly saving amount prominently displayed. Delete button per goal.
Analysis	4 KPI cards at top, then pie chart and bar chart side by side, then daily trend line, then goal analysis section with alert boxes.
All Expenses	Filterable by category, sortable by date or amount. Clear-all button for resetting data. Color-coded category pills on every row.

6.2 Design Decisions and Why

- **Dark theme:** Dark backgrounds (#0f0f13) reduce eye strain and make the colored category pills pop visually. It also makes the app feel modern rather than generic.
- **Live preview before saving:** This was the most important UX decision. If the classifier gets it wrong, the user sees it immediately and can re-type. This is better than silently saving a wrong category.
- **Sidebar quick stats:** Total spent and this month's spending are visible no matter which page you're on. You never have to navigate away to check your overall status.
- **Color-coded categories:** Each category has a consistent color used everywhere — pills, chart slices, bar chart bars. Once you learn that green = Food and blue = Transport, reading the charts is instant.

6.3 One Challenge with Streamlit

Streamlit rerenders the entire page on every interaction. This meant I had to be careful about state: if a user typed in the input box and nothing else was cached, the rerender would clear the input.

I solved this by using st.session_state for the input field and making sure the expense list was always freshly loaded from CSV rather than stored in memory. This also meant data was always consistent — another user opening the same app in a different tab would see the same data.

7. Results & Key Findings

After completing the project, I ran a structured test with 25 sample expense entries covering all 6 categories, set a goal, and evaluated every feature. Here are the key findings:

7.1 Classifier Accuracy

Category	Tested	Correct	Notes
Food	8	8 (100%)	All major food apps and items classified correctly
Transport	5	5 (100%)	All ride apps, metro, petrol classified correctly
Entertainment	4	3 (75%)	"amazon prime" initially failed; fixed by adding keyword
Essentials	4	3 (75%)	"gym" initially → Other; added keyword, retested → correct
Shopping	3	3 (100%)	Amazon, Myntra, Flipkart all correct
Overall	25	**23 (92%)**	After one round of keyword list improvements

7.2 Goal Alert Accuracy

- Rule 1 (overspending) — fired correctly in all 4 test scenarios
- Rule 2 (on track) — fired correctly in all 4 test scenarios
- Rule 3 (category overspend) — fired correctly but was found to be sensitive to small saving targets

7.3 Performance

The app loads in under 2 seconds on a standard laptop. Adding an expense takes under 0.1 seconds (CSV append). Loading and analysing 100 expenses takes under 0.3 seconds. No performance issues were observed during testing.

7.4 What Worked Better Than Expected

Positive Surprise

The live preview card — showing the classification result as you type — turned out to be the most useful feature.

During testing, it caught 3 misclassifications before they were saved.

Without it, those errors would have silently corrupted the category breakdown charts.

This taught me that transparency in AI systems is not just an ethical nice-to-have — it has real practical value.

7.5 What Did Not Work as Well

Limitation Found

The classifier has no memory of context. "Amazon" always → Shopping, even if the user types "amazon prime video subscription".

The only fix for this is phrase-level matching or a trained model — keyword scoring is inherently word-level.

Additionally, numbers embedded in descriptions (e.g. "Flat 301 maintenance") can confuse the amount parser.

8. AI/ML Concepts Demonstrated

This section maps the project's implementation directly to the AI/ML concepts covered in the course.

8.1 Classification

Classification is the task of assigning an input to one of a fixed set of categories. In this project, the input is an expense description and the categories are Food, Transport, Entertainment, Essentials, Shopping, and Other.

The classifier.classify() function implements this using keyword scoring — a deterministic variant of the Bag-of-Words approach used in probabilistic classifiers like Naive Bayes. The mathematical intuition is identical: represent text as word presence, aggregate scores per class, select the class with the highest score.

> *"A rule-based classifier can be thought of as a Naive Bayes classifier where the weights are manually set by a domain expert rather than learned from data. The prediction rule — argmax over classes — is the same."*

8.2 Statistical Analysis

The analyzer module computes descriptive statistics: sum (total spending), mean (average transaction), and grouped aggregates (category breakdown). These are the fundamental building blocks of data analysis — every ML pipeline begins with exactly these operations during exploratory data analysis (EDA).

The daily trend computation groups data by date and orders it chronologically — this is time-series analysis at its most basic, and it forms the foundation for forecasting models.

8.3 Rule-Based Decision System (AI Agent)

The goal_analysis() function in analyzer.py is a rule-based AI agent. It evaluates the current state (spending data, goal parameters, date) against explicit rules and generates appropriate responses (alerts).

This is a classic production system from symbolic AI: a set of if-then rules that fire when conditions are met. The pro-rated spending limit introduces temporal reasoning — the agent's decisions are conditioned on where we are in the month, not just raw totals. This is what separates it from a simple threshold check.

8.4 Summary Table

AI/ML Concept	Where in Project	Academic Connection
Classification	classifier.classify()	Bag-of-Words, Naive Bayes, argmax decision rule
Statistical Analysis	analyzer.full_summary()	Descriptive statistics, EDA, time-series
Rule-Based AI Agent	analyzer.goal_analysis()	Production systems, symbolic AI, temporal reasoning
NLP Input Parsing	classifier.parse_expense()	Tokenisation, regex-based information extraction
Data Visualisation	app.py charts	EDA visualisation, matplotlib, insight communication

9. Limitations & Future Work

9.1 Known Limitations

- **Misspelling sensitivity:** "swiggyy" or "nettflix" will not match any keyword and fall to Other. Fuzzy string matching (e.g. Levenshtein distance) would fix this.
- **Context blindness:** The classifier evaluates each word independently. "Amazon prime video" gets Shopping because "amazon" scores first, ignoring the entertainment context.
- **No income tracking:** The goal alerts work entirely on spending data. Without knowing income, the pro-rated limit uses only the goal parameters, which can be unrealistic for some users.
- **Single user, local only:** CSV/JSON storage is not designed for concurrent access or multiple devices.
- **Date range filtering:** The analysis page shows all-time data. A date range picker would make it much more useful for reviewing specific months.

9.2 What I Would Do Differently

If I were building this again or extending it, the three most impactful changes would be:

6. **Train a Naive Bayes classifier:** Collect 500–1000 labelled Indian expense descriptions and train a proper probabilistic model. This would handle misspellings and context much better than keyword matching.
7. **Add phrase matching:** Before word-level scoring, check for known multi-word phrases like "amazon prime", "google pay", "big bazaar". This alone would fix most of the current misclassification edge cases.
8. **Add a monthly budget input:** Let users set their monthly income/budget. The goal alerts would then be based on actual disposable income rather than just the saving target, making them far more accurate.

10. Conclusion

The Smart Expense Tracker set out to answer a simple question: can a Python application understand "250 swiggy" and do something intelligent with it? The answer, I found, is yes — with some caveats.

The rule-based classifier achieved 92% accuracy on my test set after one round of improvement. The statistical analysis correctly identifies spending patterns. The goal alert system fires at the right moments and provides specific, actionable feedback. The entire system runs on any laptop with two pip installs and one terminal command.

The most important thing I learned is not about code — it is about AI design. The best AI system for a given problem is not always the most sophisticated one. It is the one that is most appropriate for the constraints: data availability, explainability requirements, and the trust the user needs to place in it. For this problem, rule-based classification earned user trust in a way that a black-box model could not, because every decision is visible and correctable.

The live preview card — the feature that shows the AI's classification in real time before the user commits — was the most valuable insight from building this. It turned an opaque automatic process into a collaborative one. That, more than any accuracy metric, is what made the application feel intelligent.

> **Final Reflection**
>
> Building this project clarified something the course introduced theoretically: the difference between a classifier that is accurate and one that is trustworthy. For a personal finance tool, trustworthiness matters more. A user who understands why their "swiggy" was classified as Food will trust the system. A user who gets a wrong answer from an opaque neural network will not use the app again.

— End of Report —